Probability and Statistics Lab Project Report

Predicting the End of Maintenance of Software Dependencies: A Survival-Analytic and Sampling-Based Study of PyPI Packages

Project: depmortality

Author: Aryan Kumar

Date: 26 October 2026

Data snapshot: deps.dev 19 October 2026; data freeze 30 September 2025

Abstract

Open-source software ecosystems depend on volunteers whose attention is finite. When a package stops being maintained, every project that depends on it inherits unpatched vulnerabilities and incompatibilities. This report presents *depmortality*, a statistical engine for estimating when a PyPI dependency will stop being maintained. The work treats a package as a unit that moves through the states Active, Quiet, Revived and Failed in calendar-month time, and formalises the problem as a multi-state survival process observed under right censoring and delayed observability.

The report develops the probability model (discrete-time hazards, the Kaplan–Meier estimator, cause-specific hazards, mixture cure models, gamma frailty and conformal prediction) and shows how each concept maps onto the system. It then describes the implemented pipeline: a state machine with a provable no-look-ahead property, three dated failure triggers, identity resolution with hashed maintainer identifiers, and a two-part pilot sample of 2,000 packages (1,000 most-depended-upon and 1,000 stratified random) with a stratified Horvitz–Thompson estimator for projecting to the eligible universe of 42,046 packages.

On the pilot, 1,996 packages were analysed over 184,813 package-months. 904 packages went Quiet and 559 of them were revived at least once; 184 packages reached a dated Failed state. The design-weighted projection is 4,426 failed packages in the eligible universe (95% interval 3,650–5,201). The Quiet-to-Failed Kaplan–Meier curve falls to 0.859 by month 96 after confirmation. The cure-model, frailty, simulation and conformal components, and the held-out temporal evaluation, are specified here as the next stage of the project; their evaluation protocol is given without results.

Keywords: survival analysis, multi-state models, competing risks, Kaplan–Meier, Horvitz–Thompson estimation, mixture cure model, gamma frailty, conformal prediction, software supply chain.

1. Introduction

1.1 Motivation

A modern Python application pulls in tens to hundreds of transitive dependencies. Maintainer attention is unevenly distributed: many widely used packages are looked after by one person, and a maintainer's departure removes an entire package's security response at once. Existing risk scores for dependencies are typically heuristic 0–100 composites. They are not probabilities, they cannot be checked against what happened later, and they say nothing about the distinction between a package that is quiet because it is *finished* and one that is quiet because it has been *abandoned*.

1.2 Problem statement

For a package i observed at calendar month t, estimate the probability that it has failed (stopped being maintained) by month $t + h$ for horizons h of 12, 24 and 36 months, together with a lower bound on its remaining maintained life that holds with a stated finite-sample coverage. At the project level, the target is the distribution of the number of failures in a dependency tree, which requires modelling dependence between packages that share a maintainer.

1.3 Objectives

- Define maintenance states and failure events precisely enough to be computed from public data, with every threshold held in configuration.

- Build a leak-free dataset: every feature is a function of an explicit *as-of* date, and the temporal holdout is fixed in advance.
- Estimate the scale of the phenomenon on a pilot sample and project it to the eligible universe with an explicit sampling design and a variance estimate.
- Specify the statistical models (mixture cure, shared gamma frailty, whole-tree Monte Carlo, conformal lower bounds) and an evaluation protocol that cannot be tuned on test outcomes.

1.4 Scope

Version 1 covers the PyPI ecosystem only. The work reported here is Phase 0 of the project: data acquisition, label construction, the pilot study and its feasibility report. Risk is reported per package; no ranking of individuals is produced, and released tables contain only salted hashes of maintainer identities. Table 1 separates what is implemented and executed from what is specified for later phases.

Table 1. Status of project components.

Component	Status
Data pullers (deps.dev, PyPI, GitHub, git, GH Archive, OSV), caching, query-cost guard	Implemented and executed
State machine with as-of and retrospective views; bot filtering; identity resolution	Implemented, unit- and property-tested, executed
Failure triggers (a) archived/deprecated, (b) unpatched advisory, (c) Python support lapse	Implemented and executed
Pilot selection, Kaplan–Meier summaries, Horvitz–Thompson projection, Gate 1 report	Implemented and executed (Section 7)
100-package audit of states, triggers and bot filter	Audit sheet generated; scoring pending
Trigger (d): explicit “no longer maintained” statements	Specified; extractor not implemented
Mixture cure model, gamma frailty, NumPyro estimation	Specified (Sections 2.5, 2.6, 3.7); not yet estimated
Whole-tree Monte Carlo and conformal lower bounds	Specified; not yet implemented

2. Probability and Statistical Background

2.1 Discrete-time survival analysis

Let T be the (integer) number of months from a package's first PyPI release until failure, a non-negative integer random variable. The survival function and discrete hazard are

$$S(t) = P(T > t), \quad h(t) = P(T = t \mid T \geq t) \tag{1}$$

and they are linked by the product form

$$S(t) = \prod_{s=0}^{t} (1 - h(s)). \tag{2}$$

In practice T is observed with right censoring: for each package we see $Y = \min(T, C)$ and the indicator $\delta = 1\{T \leq C\}$, where C is the end of follow-up (the data freeze) or the time at which another event removes the package from risk. Inference is valid when censoring is non-informative given the model, i.e. the censoring mechanism does not carry information about T beyond what is conditioned on.

2.2 The Kaplan–Meier estimator and Greenwood variance

With distinct event times t_j, d_j events at t_j and n_j packages at risk just before t_j, the Kaplan–Meier (1958) estimator is

$$\hat{S}(t) = \prod_{j\,:\,t(j) \le t} (1 - d_j / n_j), \tag{3}$$

with Greenwood's (1926) variance estimate

$$\mathrm{Var}[\hat{S}(t)] \approx \hat{S}(t)^2 \sum_{j\,:\,t(j) \le t} d_j / (n_j (n_j - d_j)). \tag{4}$$

Its log-log transform gives confidence bands that stay within [0, 1]. This estimator is used in the project to describe the time to first Quiet and the time from Quiet to Failed. Because it is computed on the pilot without design weights, it describes the pilot, not the universe (Section 3.6).

2.3 Multi-state processes and competing risks

A package's history is a multi-state process on {Active, Quiet, Revived, Failed}. From Quiet the package can fail or be revived; these are competing causes. The cause-specific hazard for cause k is

$$\lambda_k(u) = \lim_{\Delta \to 0} P(u \le U < u + \Delta, \text{cause} = k \mid U \ge u) / \Delta, \tag{5}$$

and the probability of the first event being of cause k by time u is the cumulative incidence

$$F_k(u) = \int_0^u S(v)\, \lambda_k(v)\, dv, \qquad S(v) = \exp(-\textstyle\sum_k \Lambda_k(v)). \tag{6}$$

A Kaplan–Meier curve for a single cause that treats the competing cause as censoring estimates the *net* (cause-removed) survival, not the observed probability of that outcome. The project reports the Quiet-to-Failed curve with revival treated as censoring and labels it accordingly; the competing-risks (cumulative-incidence) analysis is part of the model phase.

2.4 Stratified sampling and the Horvitz–Thompson estimator

Let the finite universe of N eligible packages be partitioned into strata $h = 1,\ldots,H$ of sizes N_h. A stratified sample draws n_h units from stratum h. For a binary outcome y_i (failed or not), the Horvitz–Thompson (1952) estimator of the population total is the inverse-inclusion-probability sum

$$\hat{T} = \textstyle\sum_{i \in s} y_i / \pi_i = \sum_h N_h \bar{y}_h, \qquad \pi_i = n_h / N_h, \tag{7}$$

which is unbiased for the true total. When $r_h \le n_h$ units in a stratum respond (here: are successfully pulled and analysed), the project uses the responders' stratum mean $\bar{y}_h = r_h^{-1} \sum y_i$, which assumes data are missing at random within strata. The variance estimate, with finite-population correction, is

$$\mathrm{Var}(\hat{T}) = \textstyle\sum_h N_h^2 (1 - r_h / N_h)\, s_h^2 / r_h, \qquad s_h^2 = (r_h - 1)^{-1} \sum (y_i - \bar{y}_h)^2, \tag{8}$$

and a normal-approximation 95% interval is $\hat{T} \pm 1.96\ \sqrt{\mathrm{Var}(\hat{T})}$. Strata with a single responder contribute no variance because s_h^2 is undefined; the report counts them so that the resulting understatement is visible (Cochran, 1977).

2.5 Mixture cure models

Standard survival models assume every unit eventually experiences the event. For packages that are *finished* — complete, stable, no longer needing changes — a quiet period does not signal eventual failure. The mixture cure model (Farewell, 1982) represents this by a latent "cured" fraction. With u the time since going Quiet, covariates x for the susceptible-time model and z for the incidence part,

$$S(u \mid z, x) = \pi(z) + (1 - \pi(z))\, S_u(u \mid x), \qquad \pi(z) = 1 / (1 + \exp(-\gamma^T z)). \tag{9}$$

Here $\pi(z)$ is the probability that a Quiet package is finished and S_u is the survival function of the susceptible (abandoned) group. The plateau of S at $\pi(z)$ as u grows is what separates the two groups, so the model is identifiable only with sufficiently long follow-up after Quiet onset; this is why the Quiet-to-Failed curve is examined for flattening at long follow-up (Section 7.4). Classifier signals that independently suggest "finished" (Development Status 6 – Mature) are reserved for validation and never used as training labels, otherwise validation would be circular.

2.6 Shared gamma frailty

Packages maintained by the same person fail for correlated reasons (the frailty idea of Vaupel et al., 1979). A shared frailty Z_m multiplies every hazard of maintainer m's packages. With $Z_m \sim$ Gamma(shape $1/\theta$, rate $1/\theta$), so that $E[Z] = 1$ and $\mathrm{Var}(Z) = \theta$, the hazard of package i is

$$\lambda_i(t \mid Z_m) = Z_m \, \lambda_0(t) \exp(\beta^T x_i(t)). \tag{10}$$

Conditionally on Z_m the packages are independent, so the probability that none of maintainer m's packages has failed by horizon H is $E[\exp(-Z \sum \Lambda_i(H))]$, where Λ_i is the cumulative hazard without frailty. The Laplace transform of the gamma distribution, $E[e^{-sZ}] = (1 + \theta s)^{-1/\theta}$, gives the closed form

$$P(\text{no failure by H}) = (1 + \theta \textstyle\sum_i \Lambda_i(H))^{-1/\theta} \;\to\; \exp(-\textstyle\sum_i \Lambda_i(H)) \;\text{ as } \theta \to 0. \tag{11}$$

This identity is the analytic check that the planned simulator must reproduce within Monte Carlo error. The dependence it induces between two packages has Kendall's rank correlation $\tau = \theta / (\theta + 2)$ (Oakes, 1989), which gives θ an interpretable scale: $\theta = 0$ is independence and τ approaches 1 as θ grows.

2.7 Conformal prediction for survival lower bounds

Conformal prediction wraps any predictive model in a procedure whose coverage holds in finite samples without distributional assumptions beyond exchangeability. For survival times, Candès, Lei and Ren (2023) construct a lower prediction bound $\hat{L}(x)$ with

$$P(\, T \geq \hat{L}(X) \,) \geq 1 - \alpha, \tag{12}$$

calibrated on held-out data and adjusted for censoring by weighting. Here exchangeability between a 2022 calibration period and 2023+ test periods is not guaranteed because the ecosystem drifts. The project therefore plans two drift fixes for comparison: calibration on the most recent cohort only, and adaptive conformal inference (Gibbs and Candès, 2021), which updates the working miscoverage level online by

$$\alpha_{t+1} = \alpha_t + \gamma \, (\alpha - \mathrm{err}_t), \qquad \mathrm{err}_t = 1\{T_t < \hat{L}_t(X_t)\}. \tag{13}$$

2.8 Immortal-time bias and the no-look-ahead principle

If a covariate is defined using information from after the start of follow-up (for example "has made a major release"), units with that status must by construction have survived long enough to earn it, which biases estimates in their favour (Suissa, 2008). Two rules guard against this. First, covariates are time-varying and enter through counting-process records (start, stop] (Andersen and Gill, 1982). Second, every feature function f takes an explicit date s and must satisfy

$$f(D, s) = f(D_{\leq s}, s) \;\text{ for every dataset } D, \tag{14}$$

where $D_{\leq s}$ keeps only rows timestamped on or before s. This invariance is checked in the test suite by injecting future rows and asserting that the output is unchanged.

3. Mathematical Formulation of the Project

3.1 Time, origin and the activity panel

Calendar months are mapped to integers by month(d) = 12·year(d) + month(d) − 1. The time origin o_i of package i is its first PyPI upload. For each month m between o_i and the data freeze F_{end}, the activity indicator is

$$A_{im} = 1 \text{ if package i has maintainer activity in month m, else 0.} \quad (15)$$

Maintainer activity is any of: a non-bot commit on the default branch (committer date), a PyPI release, or a reply or review on an issue or pull request by an account with GitHub author association OWNER, MEMBER or COLLABORATOR. Where the association is not recorded (it is absent from GH Archive payloads before September 2017 and after September 2025), an as-of-aware fallback applies: the replier is the repository owner, or had earlier pushed to the repository or carried a maintainer association on an earlier date. The origin month counts as activity. Bot accounts are removed before any activity is computed; bot-only activity never counts as a revival.

3.2 The state machine

Let W be the quiet window (12 months, from configuration), and let L be the month of a package's last activity at or before month t. The package is **Quiet** when no activity has occurred for W consecutive months:

$$\text{Quiet at t} \Leftrightarrow t - L(t) \geq W. \quad (16)$$

Two time stamps are attached to a Quiet spell and must be kept separate. The transition is **dated** at its event month L (the last activity) but only becomes **knowable** at the observable month $L + W$. A model trained at calendar month s may use only transitions with observable month $\leq s$, otherwise the label leaks the future. Activity at month $R \geq L + W + 1$ after a Quiet spell is a **revival** (Quiet → Revived), and the package remains Revived until it goes quiet again. **Failed** is absorbing and begins in the month F of the first dated failure trigger; a Quiet spell is recognised only if it was confirmed before the failure, i.e. $L + W < F$, otherwise the transition is Active (or Revived) → Failed directly.

Two views are computed. The *as-of* view at month M uses only data up to M and, by construction, never changes when later data are added (this is the view features and training labels use). The *retrospective* view uses all data up to the freeze and dates Quiet from $L + 1$; it is used for display and for the manual audit, never for features.

3.3 Failure triggers

A package's failure time is the earliest of the following dated triggers, each of which records the date on which it becomes knowable.

- **(a) Archived or deprecated.** The GitHub repository is archived (dated by the archive timestamp), or PyPI marks the project archived or deprecated (a PEP 792 status marker, which carries no date and so cannot be placed on the timeline).
- **(b) Unpatched advisory.** An OSV advisory affecting the package has no fixed, non-yanked release within d = 90 days of publication. The failure date is publication + d; advisories whose window has not closed by the freeze are censored.
- **(c) Python support lapse.** For a CPython release $X.Y$ shipped on date r, the package fails at r + 12 months if positive evidence shows no support: a *requires_python* upper cap excluding $X.Y$, or

compiled version-specific wheels with none built for *X.Y*. Evidence of prior Python 3 support is required, so packages that never targeted it cannot fire.

- **(d) Explicit statement** ("no longer maintained"). Specified but out of scope for Phase 0; a stub marks the extension point.

The classifier *Development Status :: 7 – Inactive* is recorded as a candidate signal and its agreement with the triggers is reported, but it does not itself cause failure.

$$T_i = \min(\tau_a, \tau_b, \tau_c), \quad \delta_i = 1\{T_i \le F_{end}\}. \tag{17}$$

3.4 Identity resolution and privacy

Maintainer identity is needed to define who is active and, in later phases, which packages share a maintainer. Identifiers (GitHub logins, commit e-mail addresses, PyPI accounts) are treated as nodes of a graph and merged by union-find: an edge joins a no-reply e-mail address to the login it encodes, an e-mail to the login the GitHub API associates with a commit, and a PyPI account to an identical login seen in the repository. Generic addresses are scoped to their repository so that they cannot merge unrelated people. The person identifier of a component is then

$$\text{person}_{id} = \text{HMAC-SHA256}(\text{salt}, \text{min key in component}), \tag{18}$$

with the secret salt supplied through the environment. Processed tables are checked before writing to contain no e-mail-like strings and no identity columns.

3.5 Sample design for the pilot

The eligible universe consists of PyPI packages with at least one direct dependent in deps.dev, a resolvable non-fork GitHub repository, and first release between 2012-01-01 and 2022-12-31. The pilot has two parts: a **census** of the 1,000 packages with the most direct dependents, and a **stratified random sample** of 1,000 from the remainder. Tail strata cross the direct-dependent count (bins with edges 1, 2, 5, 20, 100) with the first-release year; allocation is proportional to stratum size, and a seed in configuration fixes the draw. The inclusion probability of a unit in stratum h is $\pi_h = n_h/N_h$ for the tail and 1 for the census, so the design weight is $w_h = 1/\pi_h$ and the stratum population size is recovered as $N_h = w_h\, n_h$.

3.6 Projection to the universe

Failure is an indicator y_i of whether package i has a dated failure by the freeze. The projected number of failed packages in the eligible universe is the estimator of equation (7) with the variance of equation (8). Its stated assumptions are: (i) packages that could not be pulled resemble pulled packages within the same stratum; (ii) the eligible universe is today's, so selection on current dependents introduces a survivor bias that is logged as a limitation; (iii) the failure definition is the Phase 0 definition (triggers a–c, undated triggers excluded).

3.7 Specified model (next phase)

The model phase combines the elements of Section 2 into a single stochastic specification, in which $x_i(t)$ are time-varying covariates evaluated as of t:

$$\lambda^{AQ}{}_i(t) = Z_{m(i)} \cdot \lambda^{AQ}{}_0(t) \cdot \exp(\beta^T x_i(t)), \quad Z_m \sim \text{Gamma}(1/\theta, 1/\theta) \tag{19}$$

$$S^{QF}(u \mid z, x) = \pi(z) + (1 - \pi(z))\, S_u(u \mid x), \quad \lambda^{QR}(u)\text{: competing revival hazard} \tag{20}$$

Estimation proceeds in two steps: a frequentist pass (Cox-type (Cox, 1972) and cure fits with cross-checks in R) and a Bayesian pass in NumPyro with piecewise-exponential baselines (the Poisson trick on person-month data), the frailty as a random effect, and the cure indicator as a latent Bernoulli variable. For a package of current age a, future lifetimes are simulated conditional on survival to a by the inverse-transform $T = S^{-1}(U \cdot S(a))$ with U uniform on (0,1) (left truncation); packages sharing a maintainer share one draw of Z per run. These components are specified and tested analytically through equation (11), but not yet estimated on data.

4. System Architecture and Implementation

4.1 Overview

The system is a Python 3.12 package with a source layout, managed by uv, with Polars and Parquet for tabular work, Pydantic models for all configuration, and httpx for web access. It is organised in four layers, each of which can be run and tested without the layers above it.

Table 2. Software layers.

Layer	Responsibility	Main modules
Utilities	Typed configuration, seeding, redacting logger, disk cache keyed by (source, query hash, snapshot date), rate-limited HTTP client, guarded BigQuery client	utils/, config.py
Pullers	deps.dev (BigQuery), PyPI JSON API, GitHub GraphQL, commit-only git clones, GH Archive via a public ClickHouse mirror, OSV bulk export	data/
Labels	Bot filter, identity resolution, activity panel, state machine, failure triggers	labels/
Build and reports	Chunked panel construction, hand-check sheet, Kaplan–Meier figures, Horvitz–Thompson projection, Gate 1 report	build.py, reports/

4.2 Data sources

Table 3. External data sources.

Source	Used for
deps.dev (BigQuery public dataset)	Package-to-repository mapping, direct-dependent counts, first upload; defines the universe
PyPI JSON API and project roles	Release history, upload dates, yanked flags, requires_python, wheel tags, status markers, owner accounts
GitHub GraphQL API	Repository existence, fork and archived flags, archive date, creation date, commit author logins
Commit-only git clones (blobless, trees omitted)	Default-branch commit history with author and committer dates
GH Archive (public ClickHouse mirror)	Issue, pull-request, review, push and release events with author association
OSV bulk export for PyPI	Security advisories with fixed versions and publication dates

The ClickHouse mirror replaces a multi-terabyte BigQuery scan of GH Archive; its event counts were compared with BigQuery on four sampled days and matched exactly on three of them and to within

0.03% (pull-request events) on the fourth. The OSV bulk export is used because the OSV query API was unreachable from the development network.

4.3 Reproducibility and safeguards

- **Query-cost guard.** Every BigQuery query is dry-run first and its estimate logged; each job carries a maximum-bytes-billed limit; any single query above 50 GB needs a recorded approval; a cumulative phase budget of 50 GB is enforced. The pilot billed 22.5 GB.
- **Caching.** Every external response is stored under the snapshot date and query hash; re-runs never refetch unless asked, and a cache-only mode rebuilds without network access.
- **Seeding and configuration.** The pilot draw, hand-check sample and any later random operation take explicit seeds from configuration. Split dates, thresholds and bot lists exist only in YAML files.
- **Memory-bounded processing.** Commit histories are parsed into columnar frames per repository and streamed to disk; the panel is built over chunks of 100 packages with identity merging done once on distinct identifier keys. A test asserts that a chunk size of 1 gives tables identical to a single chunk.
- **Testing.** The suite (over 130 tests) covers the state machine, including hypothesis properties asserting that the as-of view ignores future data, trigger logic, identity merging, parsers on trimmed real responses, the cache and the cost guard, and an end-to-end build on synthetic packages.

4.4 Workflow

The pipeline is a sequence of stages, each idempotent given the cache: **dryrun** (estimate and log all query costs), **universe** (deps.dev queries), **resolve** (GitHub repository metadata), **select** (pilot draw), **pull** (PyPI, git, GitHub logins, OSV, GH Archive events), **build** (panel, states, transitions, triggers) and **report** (figures, projection, audit sheet, Gate 1 report). All outputs of the report stage are generated by code from the processed tables; no number in the report is entered by hand.

5. Methodology

5.1 Data window and holdout

The temporal design fixes in advance, in configuration, the periods used for training (observable up to 2021-12-31), calibration (outcomes in 2022) and test (2023-01-01 to the data freeze). The data freeze was moved from the pull date to 2025-09-30 because GH Archive stopped recording author association after October 2025 and lost most comment events in mid-2026; after that date maintainer replies would be under-counted and packages would appear falsely Quiet. The test period is therefore 33 months.

5.2 Construction of labels

For every analysed package, activity months are assembled from commits, releases and qualifying replies after bot removal, the state machine is run to produce the as-of and retrospective state paths and the list of dated transitions, and the first dated trigger sets the failure month. Packages whose git history could not be cloned are excluded and reported as pull failures, not analysed with commits missing, since missing commits would manufacture false Quiet spells.

5.3 Descriptive and inferential analysis in Phase 0

Phase 0 uses three tools: counts of transitions by trigger and by period; the Kaplan–Meier estimator for the time to first Quiet (from origin) and for the time from Quiet confirmation to failure; and the stratified Horvitz–Thompson projection. Time to Quiet is measured to the dated event month and censored at the last month whose Quiet onset is still confirmable. Time from Quiet to Failed is measured from confirmation ($L + W$), because failures within the W-month window are by the state rule Active → Failed and measuring from L would put a structural zero in the first W months.

6. Evaluation Protocol

The following protocol is fixed before any model is fitted. Nothing is tuned on test-period outcomes; hyperparameters are chosen on the 2022 calibration year and then frozen.

6.1 Metrics

Discrimination. Harrell's concordance index (Harrell et al., 1982), computed over comparable pairs (pairs where the earlier time is an observed failure):

$$C = \sum_{i,j} 1\{T_i < T_j\}\, 1\{\delta_i = 1\}\, 1\{r_i > r_j\} \;/\; \sum_{i,j} 1\{T_i < T_j\}\, 1\{\delta_i = 1\}, \tag{21}$$

with r_i the predicted risk. **Accuracy of probabilities.** The inverse-probability-of-censoring-weighted Brier score at horizon t (Graf et al., 1999),

$$BS(t) = (1/n) \sum_i \left[\, 1\{T_i \le t, \delta_i = 1\}\, (\hat{S}(t \mid x_i))^2 / \hat{G}(T_i) + 1\{T_i > t\}\, (1 - \hat{S}(t \mid x_i))^2 / \hat{G}(t) \,\right], \tag{22}$$

where $\hat{G}$ is the Kaplan–Meier estimate of the censoring survival function, evaluated at t = 12, 24, 36 months. The Brier score is a strictly proper scoring rule (Gneiting and Raftery, 2007), so it rewards honest probabilities. **Calibration.** Reliability tables and plots of mean predicted against IPCW-observed failure probability in deciles of predicted risk, per horizon. **Coverage.** For conformal lower bounds at level $1 - \alpha$, the empirical coverage on test packages and the mean bound, compared across the plain, recent-cohort and adaptive variants.

6.2 Baselines

A model is judged against (i) the unconditional Kaplan–Meier curve, (ii) a Cox proportional-hazards model on time-varying covariates, and (iii) a no-frailty ($\theta = 0$) version of the full model. The frailty variance θ is reported with its interval, and Kendall's $\tau = \theta/(\theta + 2)$.

6.3 Label validation

Phase 0 labels are audited on a random sample of 100 pilot packages (seed fixed in configuration). For each, an auditor compares the generated month-by-month activity, the as-of and retrospective state paths, the transitions and the triggers against the public PyPI and GitHub pages, and records yes/no verdicts for the state machine, the triggers and the bot filter. The audit sheet has been generated; the scoring is the outstanding validation step and is required before the labels are used for modelling.

6.4 Integrity checks

The golden test of equation (11) must hold within Monte Carlo error before any simulated tree-level result is reported; the as-of invariance of equation (14) is a property test on every feature; and the

Mature classifier and “feature complete” READMEs are used only to validate cure-fraction estimates, never to fit them.

7. Results (Phase 0 Pilot)

All numbers in this section are produced by the report stage from the processed tables (data snapshot 2026-10-19, freeze 2025-09-30, $W = 12$ months).

7.1 Sample and data acquisition

Table 4. Pilot sample and acquisition.

Quantity	Value
Universe (≥ 1 direct dependent, GitHub repo, first release 2012–2022)	46,267 packages
Eligible after repository resolution (repo found, not a fork)	42,046 packages
Pilot sample	2,000 (1,000 top, 1,000 tail)
Analysed (PyPI and git pulled)	1,996 (99.8%)
Not analysed	4 (2 of them git clone failures)
Pilot packages sharing a repository with another pilot package	216
Commits read from git history	3.9 million
GH Archive events read	25.1 million
Package-month panel rows	184,813

Table 5. Activity totals in the panel.

Quantity (panel totals)	Count
Maintainer commits (non-bot, default branch)	3,215,990
PyPI releases	130,168
Maintainer replies on issues and pull requests	3,086,137
Maintainer pull-request reviews	1,602,268
Events attributed to bots (flagged and excluded from activity)	2,461,551
All maintainer-activity events	8,034,563

Maintainer replies appear in the panel for 1,466 of the 1,996 packages, so reply evidence is available for most of the sample, not only for packages with active issue trackers.

7.2 Transitions and failures

Across the pilot, 1,395 Quiet transitions occurred in 904 packages and 791 revivals in 559 packages. 184 packages reached a dated Failed state. 113 packages show maintainer activity after their failure date; such activity is ignored after failure by construction and is reported to expose how often a “failed” package is nonetheless touched.

Table 6. First dated failure trigger and the state a package was in when it failed.

Failure trigger	From Active	From Quiet	From Revived	Packages
Archived	31	39	7	77
Python support lapse	70	10	5	85
Unpatched advisory	20	0	2	22
Total	121	49	14	184

Table 7. All trigger firings (a package can fire more than one; undated firings cannot be placed on the timeline).

Trigger	Packages where it fired	Undated
Archived / deprecated	89	6
Unpatched advisory	24	0
Python support lapse	88	0

Table 8. Transitions by the period in which they become knowable.

Transition to	Train (observable ≤ 2021)	Calibration (2022)	Test (2023–Sep 2025)
Quiet	625	206	564
Revived	352	107	332
Failed	77	25	82

7.3 Classifier signals

Development Status :: 7 – Inactive appears on 26 packages; 7 of these failed and 9 went Quiet, and 7 of the 184 failed packages carry it, so the classifier agrees with the triggers on a small minority of failures. *Development Status :: 6 – Mature* (validation only) appears on 31 packages, of which 5 failed and 11 went Quiet.

7.4 Kaplan–Meier estimates

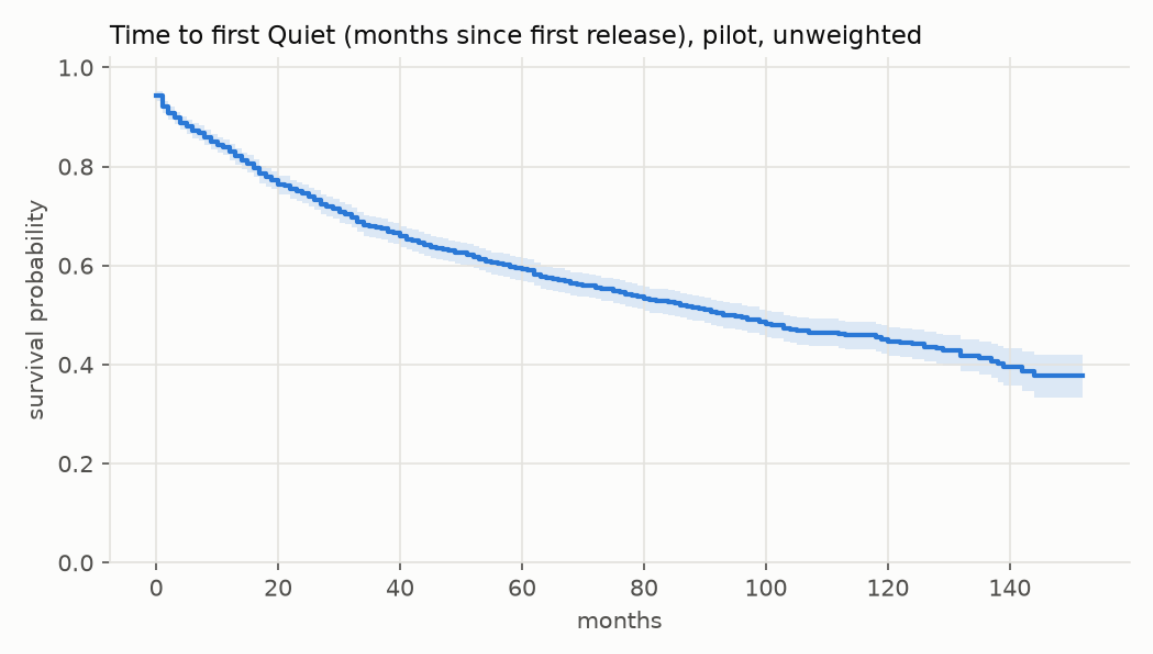


Figure 1. Kaplan–Meier estimate of time from first release to first Quiet (unweighted pilot, n = 1,996, 904 events).

Table 9. Time to first Quiet. Median 94 months; maximum follow-up 152 months.

Month after origin	$\hat{S}(t)$	At risk
12	0.830	1,651

Month after origin	Ŝ(t)	At risk
24	0.745	1,417
36	0.677	1,171
60	0.593	797
96	0.495	389

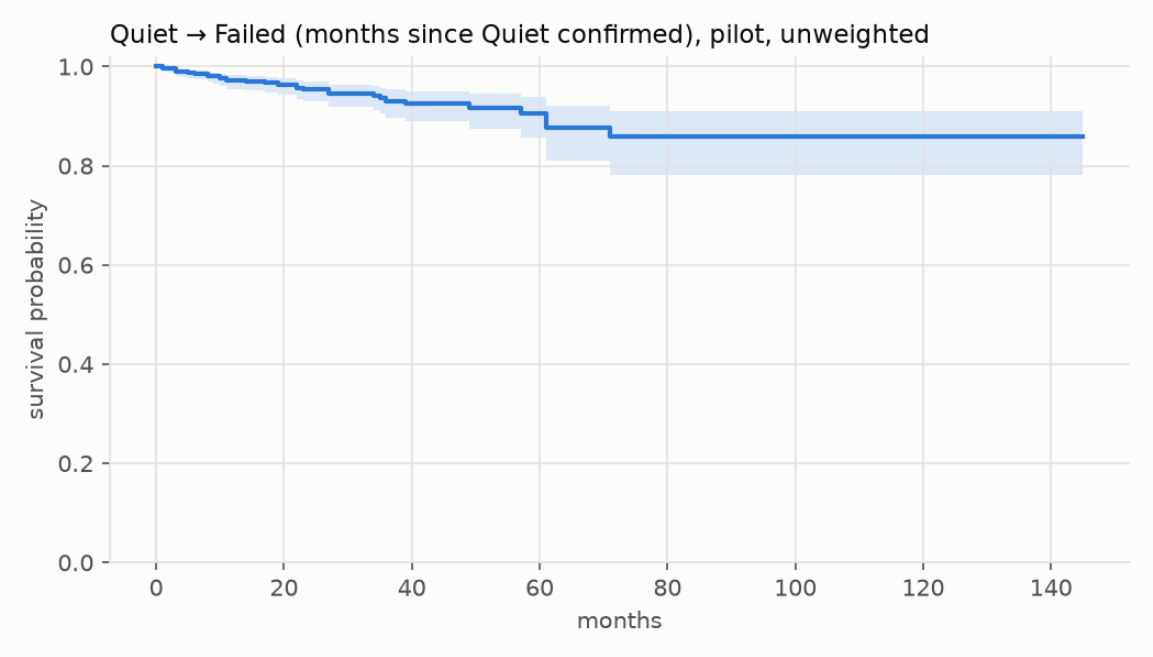


Figure 2. Kaplan–Meier estimate of time from Quiet confirmation to failure, revival treated as censoring (n = 904, 35 events).

Table 10. Quiet to Failed. The median is not reached; maximum follow-up 145 months.

Month after confirmation	Ŝ(t)	At risk
12	0.972	440
24	0.954	271
36	0.931	182
60	0.905	66
96	0.859	20

7.5 Projection to the eligible universe

The stratified Horvitz–Thompson estimator gives **4,426** failed packages over a design-weighted universe of 42,046 (standard error 396; 95% interval 3,650–5,201). Two strata with a single responder contribute no variance, so the interval is slightly optimistic. The Phase 0 feasibility threshold (at least 1,000 projected failures) is met by a wide margin, and the verdict recorded in the Gate 1 report is *viable*, provisional on the audit of Section 6.3.

7.6 Cost

The pilot ran 10 BigQuery queries billing 22.5 GB in total (28 dry runs, which are free), 859 GitHub API calls and 94 mirror queries (network calls only; cached responses excluded). No query exceeded the 50 GB single-query cap.

8. Discussion

Quiet is not terminal. More than three in five packages that went Quiet (559 of 904) were later revived, in the unweighted pilot. A model that treated Quiet as an absorbing state would therefore overstate failure, and the revival hazard is not a nuisance but an essential competing risk. This supports the multi-state formulation of Section 2.3.

Failure from Quiet is slow. The Quiet-to-Failed curve falls only to 0.859 by month 96, with 20 packages at risk, and the estimate's tail is based on few packages; its descriptive shape (a gradual decline with no sharp plateau yet visible) is not enough to estimate a cure fraction, which is why the formal test of a plateau is left to the cure-model phase. Treating revival as censoring makes this a net survival curve, and the cumulative incidence of failure from Quiet is lower than one minus this curve would suggest.

Where failures come from. Of the 184 failures, 85 are Python-support lapses and 77 are archivals, with advisories rarer at 22. Most Python-support and many archived failures occur while the package is still Active, so the failure definition captures not only abandonment but also conscious end-of-life and falling behind on compatibility. This is a measurement-validity question that the audit and the sensitivity analyses of later phases should answer, for instance by reporting results with and without trigger (c).

Reply data are a large share of activity. Maintainer replies and reviews (about 4.7 million events) outnumber maintainer commits (3.2 million) in the panel, and replies appear for 1,466 packages. Omitting GH Archive would therefore risk labelling maintainers who triage but rarely commit as Quiet. The size of this effect was not measured separately in Phase 0.

Projection. The estimate that roughly 4,400 of 42,000 eligible packages fail by the freeze is a descriptive population figure, not a forecast. It is conditional on the failure definition, the eligibility rules and the missing-at-random assumption within strata.

9. Limitations

- **Audit pending.** The 100-package check of states, triggers and bot filter has not yet been scored; label quality is therefore asserted by tests and design, not yet measured against a human reference.
- **Source coverage.** The GH Archive mirror records repository names at event time, so replies before a rename are found only where the earlier name appears in PyPI metadata and resolves to the same repository; other early replies are missed. Association data exist only from 2017-09 to 2025-09, and the freeze was shortened for this reason.
- **Survivor bias.** The eligible universe is selected on current dependents, which favours packages that survived.
- **Trigger scope.** Only triggers (a)–(c) are implemented; undated archive markers (6 firings) are excluded from timelines; trigger (c) captures compatibility lapse rather than abandonment alone.
- **Descriptive curves.** The Kaplan–Meier curves are unweighted and describe the pilot, not the universe; the net-survival interpretation under competing risks applies to the Quiet-to-Failed curve.
- **Missing data.** 19 packages lack PyPI owner data because of rate limits (identity merging for them rests on repository evidence only), and 4 packages were not analysed; they are represented by their stratum's responders under missing-at-random.

- **Model phase.** The cure, frailty, simulation and conformal components are specified, not yet fitted or validated on held-out outcomes.

10. Future Scope

- Score the audit and correct the state, trigger or bot rules where it finds systematic errors; re-run the report.
- Scale from the 2,000-package pilot to the full eligible universe, retaining the design weights.
- Fit the Phase 1 baselines: unconditional KM, Cox with time-varying covariates, cause-specific hazards for Quiet → Failed and Quiet → Revived.
- Fit the mixture cure model and test for a survival plateau after Quiet onset; validate $\pi(z)$ against the Mature classifier and feature-complete READMEs.
- Estimate the maintainer frailty θ in NumPyro and report Kendall's τ; verify the simulator against the closed form of equation (11).
- Whole-tree Monte Carlo over dependency trees with maintainer-shared draws, yielding P($\geq$ 1 critical failure), the expected number of failures and tail risk.
- Conformal lower bounds with recent-cohort calibration and adaptive conformal inference; compare their coverage under temporal drift.
- Implement trigger (d) with an audited language-model extractor, and extend to lockfile input, a command-line tool and a GitHub Action.

11. Conclusion

This work formalises dependency maintenance as a multi-state survival process with dated, knowable transitions and implements the data and labelling infrastructure for it, with leak-prevention built in as a tested property. A two-part pilot of 2,000 packages, analysed with Kaplan–Meier estimation and a stratified Horvitz–Thompson projection, shows that a substantial number of failures is observable (184 in the pilot; about 4,400 projected, 95% interval 3,650–5,201), that going Quiet is frequently followed by revival, and that failure from Quiet is slow enough that a cure-model analysis is warranted. The next steps, an audit of the labels followed by the cure, frailty, simulation and conformal models under a pre-registered temporal holdout, are specified with their formulas and evaluation protocol.

References

Andersen, P. K. and Gill, R. D. (1982). Cox's regression model for counting processes: a large sample study. *The Annals of Statistics*, 10(4), 1100–1120.

Candès, E., Lei, L. and Ren, Z. (2023). Conformalized survival analysis. *Journal of the Royal Statistical Society: Series B*, 85(1), 24–45.

Cochran, W. G. (1977). *Sampling Techniques*, 3rd ed. Wiley, New York.

Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2), 187–220.

Farewell, V. T. (1982). The use of mixture models for the analysis of survival data with long-term survivors. *Biometrics*, 38(4), 1041–1046.

Gibbs, I. and Candès, E. (2021). Adaptive conformal inference under distribution shift. *Advances in Neural Information Processing Systems* 34.

Gneiting, T. and Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359–378.

Graf, E., Schmoor, C., Sauerbrei, W. and Schumacher, M. (1999). Assessment and comparison of prognostic classification schemes for survival data. *Statistics in Medicine*, 18(17–18), 2529–2545.

Greenwood, M. (1926). The natural duration of cancer. *Reports on Public Health and Medical Subjects*, 33, 1–26. HMSO, London.

Harrell, F. E., Califf, R. M., Pryor, D. B., Lee, K. L. and Rosati, R. A. (1982). Evaluating the yield of medical tests. *JAMA*, 247(18), 2543–2546.

Horvitz, D. G. and Thompson, D. J. (1952). A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association*, 47(260), 663–685.

Kaplan, E. L. and Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 457–481.

Oakes, D. (1989). Bivariate survival models induced by frailties. *Journal of the American Statistical Association*, 84(406), 487–493.

Suissa, S. (2008). Immortal time bias in pharmaco-epidemiology. *American Journal of Epidemiology*, 167(4), 492–499.

Vaupel, J. W., Manton, K. G. and Stallard, E. (1979). The impact of heterogeneity in individual frailty on the dynamics of mortality. *Demography*, 16(3), 439–454.

Data sources: deps.dev (https://deps.dev); GH Archive (https://www.gharchive.org); OSV, Open Source Vulnerabilities (https://osv.dev); Python Package Index (https://pypi.org).

Appendix A. Configuration Parameters

Table A1. Main parameters (all held in config/*.yaml).

Parameter	Value	Meaning
quiet_window_months (W)	12	Months without maintainer activity before Quiet
advisory_window_days (d)	90	Allowed time to a fixed release after an advisory
python_support_lag_months	12	Months after a CPython X.Y.0 release before lack of support counts as failure
train_end	2021-12-31	Last observable date for training data
calibration year	2022-01-01 – 2022-12-31	Tuning and calibration outcomes
test_start	2023-01-01	First test outcome date
data_freeze	2025-09-30	Last date of any data used
cohort (first release)	2012-01-01 – 2022-12-31	Eligibility window
n_top, n_tail	1,000, 1,000	Census and stratified-sample sizes
tail dependent-bin edges	1, 2, 5, 20, 100	Stratification of direct-dependent counts
BigQuery limits	50 GB per query; 50 GB per phase	Cost guard

Appendix B. Reproducing the Pilot

From the repository root, with credentials supplied as environment variables, the stages run in order as module invocations of the pipeline: *dryrun*, *universe*, *resolve*, *select*, *pull*, *build*, *report*. The report stage writes *reports/gate1.md*, the figures, and the audit sheet *reports/gate1_handcheck.csv*. Every pull is cached under the snapshot date; *--cache-only* rebuilds from the cache without network access.